
Part A: Solving the LIF Equation with NDSolve

Step 1: Verify Your Setup

```
In[4]:= Print["Wolfram Language ready!"]
Print[$Version]
Wolfram Language ready!
14.3.0 for Linux x86 (64-bit) (July 8, 2025)
```

Step 2: Understand the LIF Equation in Wolfram

```
In[6]:= (* The LIF differential equation: *)
(* tau * V'[t] == -(V[t] - Vrest) + R * I[t] *)
(* *)
(* tau = membrane time constant (seconds) *)
(* Vrest = resting potential (mV) *)
(* R = membrane resistance (MOhm) *)
(* I[t] = input current (nA) *)
(* *)
(* When V[t] ≥ Vthresh, neuron spikes and resets *)
```

Step 3: Define Neuron Parameters

```
In[7]:= (* ===== Neuron Parameters ===== *)
tau = 0.020; (* Membrane time constant: 20 ms *)
Vrest = -70.0; (* Resting potential in mV *)
Vthresh = -55.0; (* Spike threshold in mV *)
Vreset = -75.0; (* Reset potential after spike *)
R = 1.0; (* Membrane resistance, normalized *)
(* ===== Input Current ===== *)
(* Constant current that turns on at t=0.05 and off at t=0.4 *)
Iinput[t_] := If[0.05 ≤ t ≤ 0.4, 18.0, 0.0];
Print["Parameters defined."]
Parameters defined.
```

KNOWLEDGE CHECK

1. The input current is defined as a function of time using `If[]`. In plain English, what does `Iinput[t]` return at $t = 0.03$? At $t = 0.2$? At $t = 0.5$?

At $t = 0.03$, the condition $0.05 \leq t \leq 0.4$ is false, so `Iinput[t]` returns 0.0, meaning no current is applied yet. At $t = 0.2$, the condition is true, so `Iinput[t]` returns 18.0, meaning the neuron receives input

current. At $t = 0.5$, the condition is false again, so `linput[t]` returns 0.0 because the stimulus has already turned off.

2. Why do we use `:=` (delayed assignment) for `linput` instead of `=` (immediate assignment)?

We use `:=` (delayed assignment) so the function is evaluated each time it is called with a value of t . This allows the input current to change correctly during the simulation as time progresses.

Step 4: Solve the LIF Equation

```
In[29]:= (* Solve the LIF equation with spike reset *)

spikeCount = 0;
spikeTimes = {};

sol = NDSolve[
{
tau * V'[t] == -(V[t] - Vrest) + R * Iinput[t],
V[0] == Vrest,
WhenEvent[V[t] ≥ Vthresh,
{
V[t] → Vreset,
spikeCount = spikeCount + 1,
AppendTo[spikeTimes, t]
}
}
],
V,
{t, 0, 0.5}
];

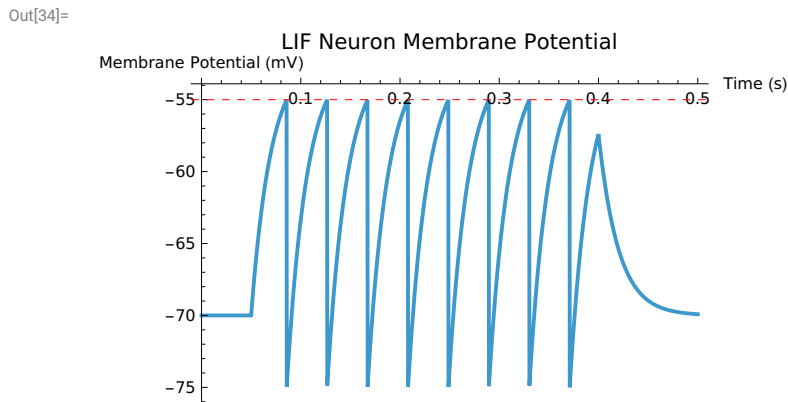
Print["Neuron fired ", spikeCount, " spikes"]
Print["Spike times: ", spikeTimes]

Neuron fired 8 spikes

Spike times:
{0.0858352, 0.126573, 0.16731, 0.208048, 0.248786, 0.289523, 0.330261, 0.370999}
```

Step 5: Plot the Membrane Potential

```
In[34]:= Plot[
  Evaluate[V[t] /. sol],
  {t, 0, 0.5},
  AxesLabel → {"Time (s)", "Membrane Potential (mV)"},
  PlotLabel → "LIF Neuron Membrane Potential",
  Epilog → {Red, Dashed, InfiniteLine[{{0, Vthresh}, {1, Vthresh}}]}
]
```



KNOWLEDGE CHECK

1. Look at the shape of V between spikes. Is it a straight line or a curve? Why?

The shape of V between spikes is a curve, not a straight line. This happens because the LIF neuron follows a differential equation where the voltage changes gradually toward a steady value rather than increasing at a constant rate. The leak term $-(V - V_{\text{rest}})$ causes the voltage to approach a limit smoothly, which produces a curved (exponential-like) rise between spikes.

2. What happens to V after the current turns off ($t > 0.4$)? Describe what you see and explain it in terms of the LIF equation.

After the current turns off ($t > 0.4$), the membrane potential drops back toward the resting potential (-70 mV) and stops spiking. This happens because the input current term $R \cdot I[t]$ becomes zero, so the LIF equation is dominated by the leak term $-(V - V_{\text{rest}})$, which pulls the voltage back toward the resting potential.

Part B: Interactive Manipulate[] Dashboard

Step 6: Build the Dashboard

```
In[35]:= Manipulate[
  Module[{sol, sc = 0, st = {}, iFunc},

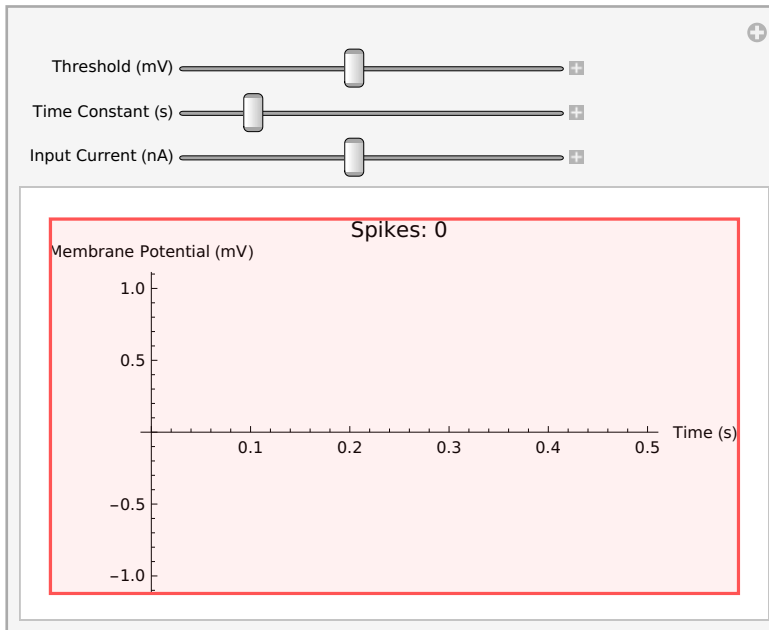
    iFunc[t_] := If[0.05 ≤ t ≤ 0.4, currentVal, 0.0];

    sol = NDSolve[
      {
        tauVal*V'[t] == -(V[t] - Vrest) + R*iFunc[t],
        V[0] == Vrest,
        WhenEvent[
          V[t] ≥ thresh,
          {
            V[t] → Vreset,
            sc = sc + 1,
            AppendTo[st, t]
          }
        ]
      },
      V,
      {t, 0, 0.5}
    ];

    Plot[
      Evaluate[V[t] /. sol],
      {t, 0, 0.5},
      AxesLabel → {"Time (s)", "Membrane Potential (mV)"},
      PlotLabel → StringForm["Spikes: ``", sc],
      Epilog → {Red, Dashed, InfiniteLine[{{0, thresh}, {1, thresh}}]}
    ]
  ],

  {{thresh, -55.0, "Threshold (mV)"}, -65, -45, 1},
  {{tauVal, 0.020, "Time Constant (s)"}, 0.005, 0.100, 0.005},
  {{currentVal, 18.0, "Input Current (nA)"}, 0, 40, 1}
]
```

Out[35]=



KNOWLEDGE CHECK

1. Using your dashboard, find the minimum input current needed to make the neuron fire at least one spike (with default threshold and tau). This is called the rheobase. What value did you find?

With the default threshold (-55 mV) and time constant (0.020 s), the minimum input current needed to make the neuron fire at least one spike is about 15 nA. That is the rheobase in this model, because the current has to be just large enough to raise the membrane potential from -70 mV to threshold before the stimulus turns off.

2. Set the current to 25 nA. Now slowly increase the threshold from -65 to -45 mV. At what threshold does the neuron stop firing? Why?

With the current set to 25 nA, the neuron stops firing at about -45 mV. This happens because a higher threshold means the voltage has to climb farther before a spike can occur. In this model, -45 mV is about the highest voltage the membrane can approach under that input, so once the threshold reaches that value, the neuron can no longer cross it and stops spiking.

3. Set threshold to -55 and current to 20 . Compare $\tau = 0.010$ vs $\tau = 0.050$. Which produces more spikes? Which produces a smoother membrane potential curve?

With threshold set to -55 mV and current set to 20 nA, $\tau = 0.010$ produces more spikes, while $\tau = 0.050$ produces fewer spikes. A smaller time constant lets the membrane charge and reset faster, so it

reaches threshold more often. The $\tau = 0.050$ case gives a smoother membrane potential curve because the voltage changes more slowly and gradually.

REFLECTION

The Manipulate[] dashboard lets you explore neuron behavior interactively in a way that static code cannot. Describe one insight you gained from adjusting the sliders that you did NOT get from reading the booklet. How did the visual feedback change your understanding?

One insight I gained from adjusting the sliders in the Manipulate[] dashboard was how strongly the input current affects the firing rate of the neuron. While the booklet explains that neurons fire when the membrane potential crosses a threshold, seeing the simulation made it clearer how small changes in current can dramatically change the number of spikes produced. When I increased the current slider, the neuron began firing more frequently, and when I lowered it near the rheobase, the neuron stopped firing entirely. The visual plot helped me see how the membrane potential rises faster with stronger input and resets repeatedly after each spike. This interactive feedback made the relationship between input strength, membrane potential dynamics, and spike frequency much easier to understand than reading the explanation alone. It showed how the integrate-leak-fire cycle works continuously over time, reinforcing the concept that spiking neurons accumulate input and respond dynamically rather than producing a single static output.

Part C: Firing Rate Curve (f-I Curve)

Step 7: Generate the f-I Curve

```
In[36]:= (* Generate f-I curve data *)
```

```
fiData = Table[
  Module[{sc = 0, st = {}, sol, iFunc},

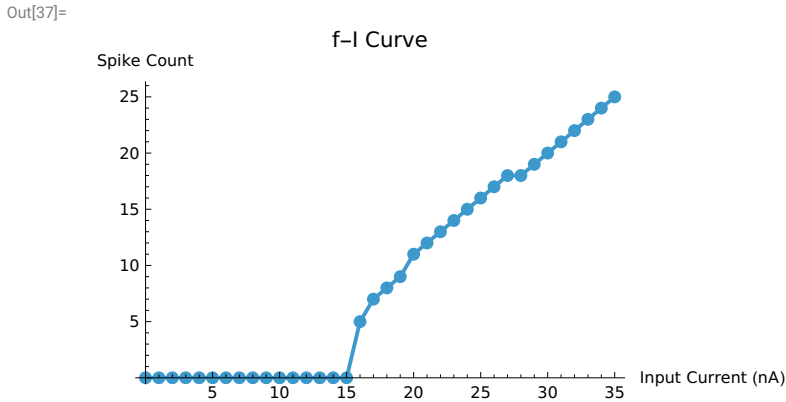
    iFunc[t_] := If[0.05 ≤ t ≤ 0.4, Icurr, 0.0];

    sol = NDSolve[
      {
        tau*V'[t] == -(V[t] - Vrest) + R*iFunc[t],
        V[0] == Vrest,
        WhenEvent[
          V[t] ≥ Vthresh,
          {
            V[t] → Vreset,
            sc = sc + 1,
            AppendTo[st, t]
          }
        ]
      },
      V,
      {t, 0, 0.5}
    ];

    {Icurr, sc}
  ],
  {Icurr, 0, 35, 1}
];
```

```
(* Plot the f-I curve *)
```

```
ListLinePlot[
  fiData,
  AxesLabel → {, },
  PlotLabel → ,
  PlotMarkers → Automatic
]
```

KNOWLEDGE CHECK

1. At what input current does your neuron first start firing? This is related to the rheobase you found with the dashboard.

The neuron first starts firing at about 15 nA, which is close to the rheobase value observed earlier using the interactive dashboard. Below this current, the membrane potential does not reach the threshold, so no spikes occur.

2. Is the f-I curve a straight line, or does it curve? What would a perfectly straight line mean about how the neuron encodes stimulus intensity?

The f-I curve is slightly curved rather than a perfectly straight line. This happens because the membrane potential increases according to the LIF dynamics, where the leak term affects how quickly the neuron approaches the threshold. A perfectly straight line would mean the neuron encodes stimulus intensity in a perfectly linear way, where every increase in input current produces a proportional increase in firing rate.

3. Real neurons often show a “saturation” effect at high currents (firing rate plateaus). Does your LIF model show this? Why or why not?

The basic LIF model does not show strong saturation at high currents. As the input current increases, the neuron continues to fire more frequently without a clear plateau. Real neurons often show saturation because of biological factors such as refractory periods, ion channel limits, and metabolic constraints, which are not included in the simple LIF model used in this simulation.

Part D: Encoding a Signal as Spikes

Step 8: Rate Coding with a Varying Signal

```
In[38]:= (* Define a sine wave input current *)
Isine[t_] := 15.0 + 12.0*Sin[2 Pi*5*t];
```

```

(* Run NDSolve with sine-wave input *)
spikeCount = 0;
spikeTimes = {};

solSine = NDSolve[
{
  tau*V'[t] == -(V[t] - Vrest) + R*Isine[t],
  V[0] == Vrest,
  WhenEvent[
    V[t] ≥ Vthresh,
    {
      V[t] → Vreset,
      spikeCount = spikeCount + 1,
      AppendTo[spikeTimes, t]
    }
  ]
},
V,
{t, 0, 1.0}
];

(* Top plot: membrane potential *)
plot1 = Plot[
  Evaluate[V[t] /. solSine],
  {t, 0, 1.0},
  AxesLabel → {, },
  PlotLabel → ,
  Epilog → {Red, Dashed, InfiniteLine[{0, Vthresh}, {1, Vthresh}]}
];

(* Bottom plot: input current *)
plot2 = Plot[
  Isine[t],
  {t, 0, 1.0},
  AxesLabel → {, },
  PlotLabel →
];

```

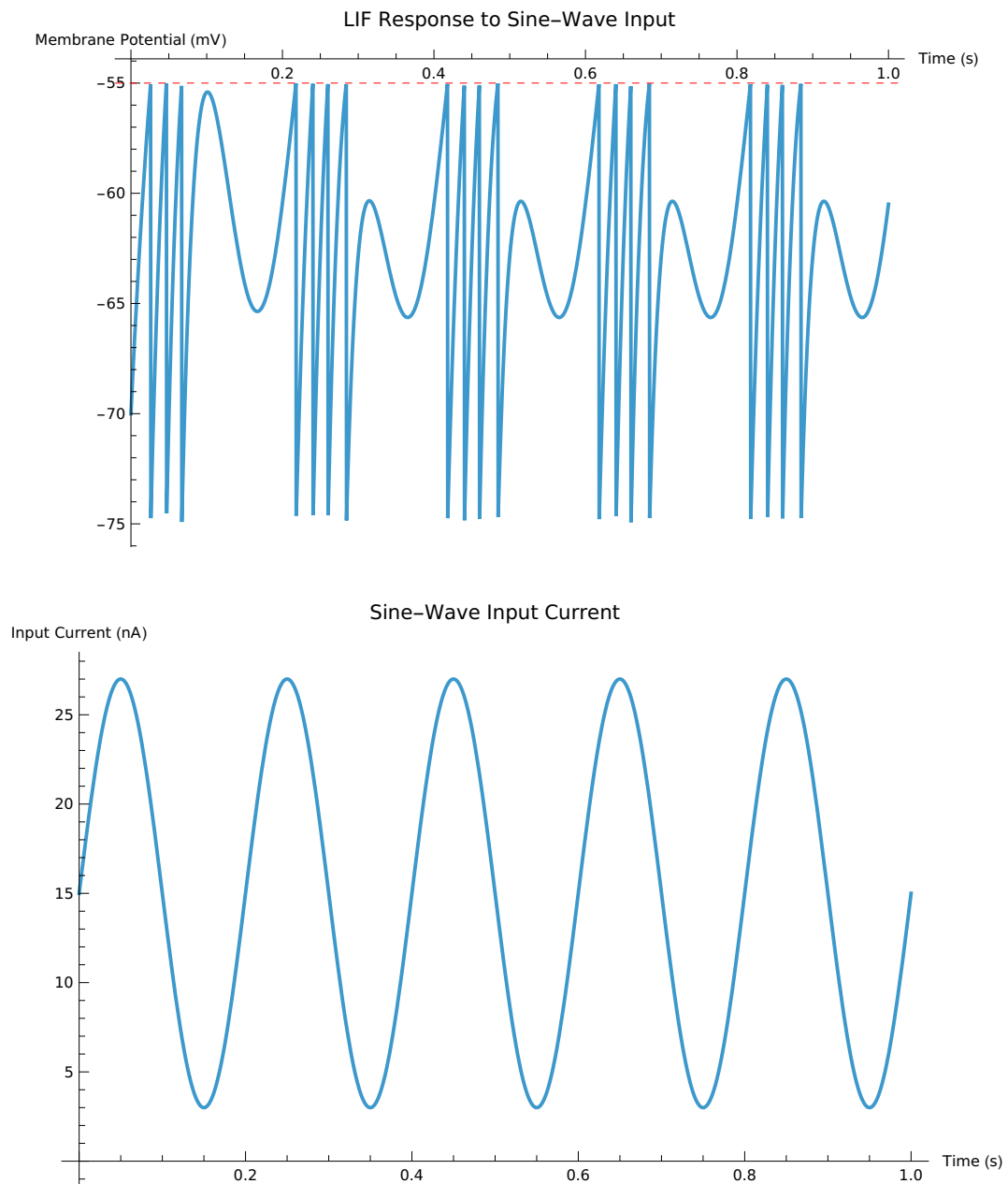
(* Show both plots *)

```
GraphicsColumn[{plot1, plot2}]
```

```
Print["Neuron fired ", spikeCount, " spikes"]
```

```
Print["Spike times: ", spikeTimes]
```

Out[44]=



Neuron fired 19 spikes

Spike times: {0.0260992, 0.0468183, 0.0671122, 0.217907, 0.240209,
0.260038, 0.284442, 0.41797, 0.440257, 0.460087, 0.484523, 0.617971,
0.640257, 0.660087, 0.684524, 0.817971, 0.840257, 0.860087, 0.884524}

REFLECTION

Look at your spike train alongside the sine wave input. The neuron is acting as an analog-to-digital converter: it takes a continuous signal and converts it into discrete spikes. How well does the spike pattern represent the original signal? What information is preserved? What is lost?

The spike train represents the overall pattern of the sine wave reasonably well, but not perfectly. When the sine wave input current is higher, the neuron fires more frequently, and when the input current is lower, the neuron fires less often or may stop firing. This means the spike train preserves important information about the strength and timing of the signal, especially the peaks and troughs of the sine wave. However, some information is lost because the spikes are discrete events rather than a continuous signal. The exact shape and smooth changes of the sine wave are not preserved, and small variations in the signal may not produce spikes if they do not push the membrane potential past the threshold. As a result, the neuron captures the general intensity pattern of the signal over time, but it loses fine details of the original waveform.

PART E: EXPERIMENTS

Experiment 1: Threshold Sweep

Using your Manipulate[] dashboard, systematically change the threshold from -65 to -45 in steps of 5. For each setting, record the spike count. Create a table or plot of threshold vs. spike count.

```

In[47]:= thresholdData = Table[
  Module[{sc = 0, st = {}, sol, iFunc},

    iFunc[t_] := If[0.05 ≤ t ≤ 0.4, 18.0, 0.0];

    sol = NDSolve[
      {
        tau * V'[t] == -(V[t] - Vrest) + R * iFunc[t],
        V[0] == Vrest,
        WhenEvent[
          V[t] ≥ thresh,
          {
            V[t] → Vreset,
            sc = sc + 1,
            AppendTo[st, t]
          }
        ]
      },
      V,
      {t, 0, 0.5}
    ];

    {thresh, sc}
  ],
  {thresh, -65, -45, 5}
];

Grid[
  Prepend[thresholdData, {"Threshold (mV)", "Spike Count"}],
  Frame → All
]

```

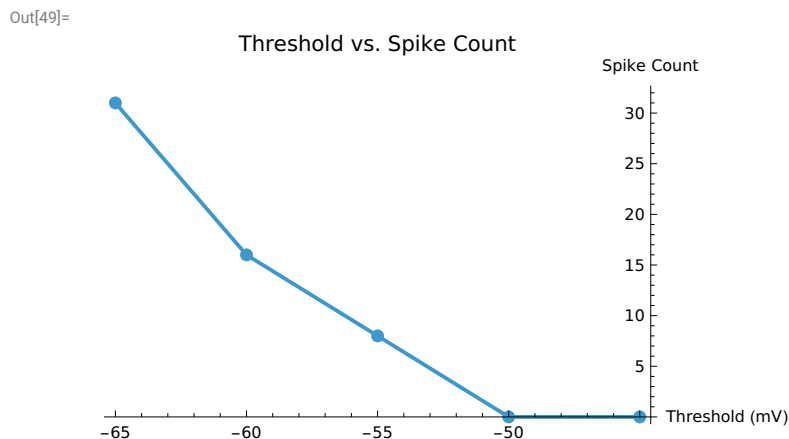
Out[48]=

Threshold (mV)	Spike Count
-65	31
-60	16
-55	8
-50	0
-45	0

```

In[49]:= ListLinePlot[
  thresholdData,
  AxesLabel → {"Threshold (mV)", "Spike Count"},
  PlotLabel → "Threshold vs. Spike Count",
  PlotMarkers → Automatic
]

```



1. What parameter did you change, and to what values?

In this experiment, I changed the threshold voltage of the neuron. The threshold values tested were –65 mV, –60 mV, –55 mV, –50 mV, and –45 mV.

2. What did you expect to happen?

I expected that increasing the threshold would reduce the number of spikes. A higher threshold means the membrane potential must rise further before the neuron fires, so the neuron should spike less often.

3. What actually happened?

The spike count decreased as the threshold increased. At –65 mV, the neuron fired 31 spikes, at –60 mV it fired 16 spikes, and at –55 mV it fired 8 spikes. When the threshold was raised to –50 mV and –45 mV, the neuron did not fire any spikes.

4. Why does this make sense given the LIF equation?

This behavior matches the LIF equation because the membrane potential must reach the threshold before a spike occurs. When the threshold is set closer to the resting potential (such as –65 mV), the neuron reaches it easily and fires frequently. As the threshold increases, the neuron needs more accumulated input current to reach it. When the threshold becomes too high (–50 mV or –45 mV), the membrane potential never reaches it, so no spikes occur.

Experiment 2: Time Constant Comparison

Generate 3 membrane potential plots side by side: $\tau = 0.010$, $\tau = 0.020$, $\tau = 0.050$. Use `GraphicsRow[{plot1, plot2, plot3}]` to display them together. Describe how the “shape” of V changes.


```

In[50]:= makePlot[tauTest_] := Module[{sc = 0, st = {}, sol, iFunc},

  iFunc[t_] := If[0.05 ≤ t ≤ 0.4, 18.0, 0.0];

  sol = NDSolve[
  {
    tauTest*V'[t] == -(V[t] - Vrest) + R*iFunc[t],
    V[0] == Vrest,
    WhenEvent[
      V[t] ≥ Vthresh,
      {
        V[t] → Vreset,
        sc = sc + 1,
        AppendTo[st, t]
      }
    ]
  },
  V,
  {t, 0, 0.5}
];

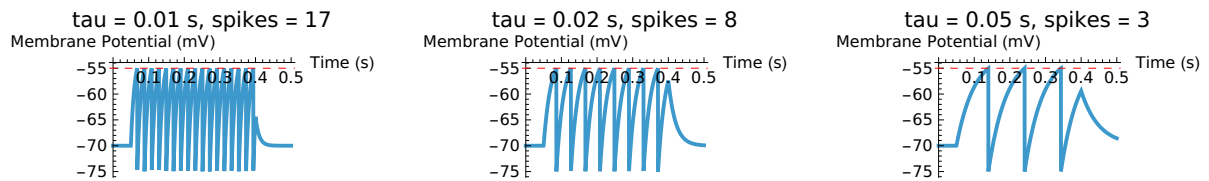
Plot[
  Evaluate[V[t] /. sol],
  {t, 0, 0.5},
  AxesLabel → {"Time (s)", "Membrane Potential (mV)"},
  PlotLabel → StringForm["tau = `` s, spikes = ``", tauTest, sc],
  Epilog → {Red, Dashed, InfiniteLine[{{0, Vthresh}, {1, Vthresh}}]}
];

plot1 = makePlot[0.010];
plot2 = makePlot[0.020];
plot3 = makePlot[0.050];

GraphicsRow[{plot1, plot2, plot3}]

```

Out[54]=



1. What parameter did you change, and to what values?

In this experiment, I changed the membrane time constant τ and tested three values: 0.010 s, 0.020 s, and 0.050 s.

2. What did you expect to happen?

I expected that a smaller time constant would let the membrane potential respond more quickly to the input current, which would produce more spikes. I also expected that a larger time constant would make the voltage change more slowly and create a smoother curve with fewer spikes.

3. What actually happened?

The results matched my expectation. When $\tau = 0.010$, the neuron fired 17 spikes. When $\tau = 0.020$, it fired 8 spikes. When $\tau = 0.050$, it fired only 3 spikes. The membrane potential curve for the smaller τ rose and reset more quickly, while the larger τ produced a slower, smoother voltage curve.

4. Why does this make sense given the LIF equation?

This makes sense because in the LIF equation, τ controls how quickly the membrane potential changes over time. A smaller τ means the voltage can rise faster in response to input current, so the neuron reaches threshold more often and fires more spikes. A larger τ slows down the voltage changes, so the neuron takes longer to reach threshold and fires less often. This is why the larger τ also makes the membrane potential plot look smoother and more gradual.

Experiment 5: Two Different f-I Curves

Generate f-I curves for two different time constants (e.g., $\tau = 0.010$ and $\tau = 0.040$) and plot them on the same graph. How does τ affect the neuron's sensitivity to input current?

```
In[55]:= fiData1 = Table[
  Module[{sc = 0, st = 0, sol, iFunc},

    iFunc[t_] := If[0.05 ≤ t ≤ 0.4, Icurr, 0.0];

    sol = NDSolve[
```

```

{
  0.010 * V'[t] == -(V[t] - Vrest) + R * iFunc[t],
  V[0] == Vrest,
  WhenEvent[
    V[t] ≥ Vthresh,
    {
      V[t] → Vreset,
      sc = sc + 1,
      AppendTo[st, t]
    }
  ]
},
V,
{t, 0, 0.5}
];

{Icurr, sc}
],
{Icurr, 0, 35, 1}
];

```

```

fiData2 = Table[
  Module[{sc = 0, st = {}, sol, iFunc},

    iFunc[t_] := If[0.05 ≤ t ≤ 0.4, Icurr, 0.0];

    sol = NDSolve[
      {
        0.040 * V'[t] == -(V[t] - Vrest) + R * iFunc[t],
        V[0] == Vrest,
        WhenEvent[
          V[t] ≥ Vthresh,
          {
            V[t] → Vreset,
            sc = sc + 1,
            AppendTo[st, t]
          }
        ]
      },

```

```

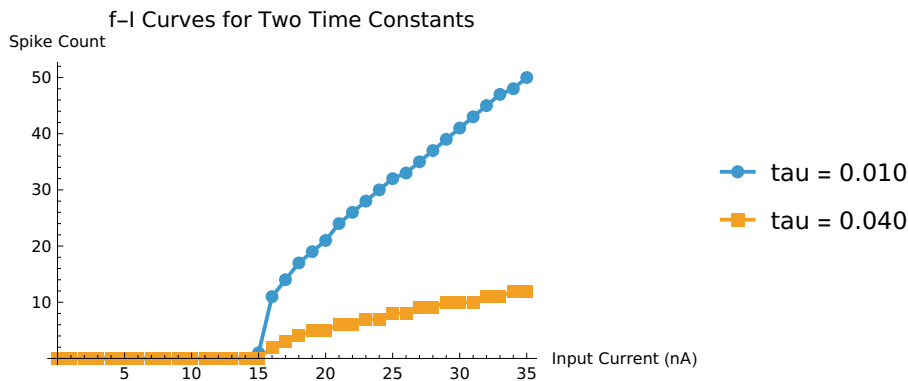
V,
{t, 0, 0.5}
];

{Icurr, sc}
],
{Icurr, 0, 35, 1}
];

ListLinePlot[
{fiData1, fiData2},
AxesLabel → {"Input Current (nA)", "Spike Count"},
PlotLabel → "f-I Curves for Two Time Constants",
PlotLegends → {"tau = 0.010", "tau = 0.040"},
PlotMarkers → Automatic
]

```

Out[57]=



1. What parameter did you change, and to what values?

In this experiment, I changed the membrane time constant τ and compared two values: $\tau = 0.010$ s and $\tau = 0.040$ s. For each value of τ , I generated an f-I curve by sweeping the input current from 0 to 35 nA and counting how many spikes occurred.

2. What did you expect to happen?

I expected that the neuron with the smaller time constant ($\tau = 0.010$) would produce more spikes because the membrane potential can respond faster to the input current. I expected the neuron with $\tau = 0.040$ to respond more slowly and therefore produce fewer spikes.

3. What actually happened?

The results matched the expectation. The $\tau = 0.010$ neuron produced many more spikes as the input current increased, reaching about 50 spikes at 35 nA. The $\tau = 0.040$ neuron produced far fewer spikes,

reaching only about 12 spikes at the same current. The smaller τ also caused the neuron to begin firing at a slightly lower current than the larger τ .

4. Why does this make sense given the LIF equation?

In the LIF equation, τ controls how quickly the membrane potential changes over time. A smaller τ allows the voltage to rise and reset faster, enabling the neuron to reach the threshold more often and fire more spikes. A larger τ slows down the voltage dynamics, so the neuron integrates input more slowly and fires less frequently. This is why the f - I curve for $\tau = 0.010$ is much steeper than the curve for $\tau = 0.040$.

Part F: Final Reflection

REFLECTION

1. Using your Manipulate[] dashboard, describe what happens as you gradually increase input current from below threshold to well above it. At what point does the neuron start firing? What does this threshold behavior tell you about how biological neurons make “decisions”?

Using the Manipulate[] dashboard, I observed that when the input current is below a certain level, the membrane potential increases slightly but never reaches the threshold, so the neuron does not fire. As I gradually increased the input current, the membrane potential rose faster and eventually crossed the threshold, causing the neuron to fire spikes. In my simulation, the neuron began firing at around 15 nA, which corresponds to the rheobase of the model. As the current increased further, the neuron fired more frequently because the membrane potential reached the threshold more quickly after each reset. This threshold behavior shows how biological neurons make “decisions.” Instead of responding proportionally to every small input, they accumulate signals over time and only produce an output when the combined input is strong enough to cross a threshold. This allows neurons to filter out weak or noisy signals and respond only when the input is significant enough.

REFLECTION

2. Examine your f - I curve. Is the relationship linear? What biological significance does the shape of this curve have for how the brain encodes stimulus intensity?

When examining the f - I curve, the relationship between input current and spike count is not perfectly linear. The neuron does not fire at all below the rheobase, and once the input current passes that point, the firing rate begins to increase as the current increases. However, the increase is gradual rather than a perfect straight line. This shape is biologically meaningful because it shows how neurons encode stimulus intensity through firing rate. Weak stimuli may not produce any spikes, but

once the input becomes strong enough, the neuron responds by firing more frequently. In the brain, this allows neurons to represent stronger signals with higher spike rates while filtering out weaker signals that do not reach the threshold.

REFLECTION

3. How does changing the membrane time constant affect firing? In biological terms, what types of neurons might have short vs. long time constants, and what would each be good at?

Changing the membrane time constant (τ) affects how quickly the membrane potential responds to input current. In the simulation, a smaller τ caused the membrane potential to rise and reset more quickly, which produced more frequent spikes. A larger τ made the membrane potential change more slowly, resulting in fewer spikes and a smoother voltage curve. In biological terms, neurons with short time constants respond quickly to inputs and are good at detecting rapid changes in signals, such as neurons involved in fast sensory processing. Neurons with long time constants integrate inputs over a longer period of time, making them better for tasks that require accumulating information, such as decision making or maintaining activity over time.

REFLECTION

4. Compare the LIF model to what you learned about real neurons in the booklet. What does the model capture well? What important features does it miss? (Think about: dendrite morphology, multiple ion channels, neurotransmitter dynamics, adaptation.)

The Leaky Integrate-and-Fire (LIF) model captures several important features of real neurons. It models how neurons integrate incoming signals over time, leak membrane potential toward a resting value, fire a spike when the threshold is reached, and then reset after firing. This reflects the basic behavior of biological neurons described in the booklet, where neurons accumulate inputs and generate an all-or-nothing spike once the membrane potential crosses a threshold.

However, the LIF model is a simplified representation and misses many important biological details. Real neurons have complex dendritic structures that influence how inputs from different synapses are processed. They also contain multiple ion channels that create different firing patterns and dynamics, which are not represented in the simple LIF equation. In addition, the model does not include neurotransmitter dynamics, synaptic delays, or mechanisms like adaptation, where a neuron becomes less likely to fire after repeated spikes. Because of these simplifications, the LIF model is useful for understanding the basic principles of neural computation, but it cannot capture the full complexity of real biological neurons.

REFLECTION

5. Wolfram's Manipulate[] gave you real-time interactive exploration. How did this change your understanding compared to just reading about neurons? What was the most surprising thing you discovered by moving the sliders?

Using Wolfram's Manipulate[] dashboard helped me understand neuron behavior much more clearly than just reading about it. When reading the booklet, the concepts of threshold, time constant, and input current are explained theoretically, but the interactive simulation allowed me to immediately see how changing these parameters affects the membrane potential and spike pattern. By moving the sliders, I could observe how the neuron transitions from not firing to firing rapidly as the input current increases. The most surprising thing I discovered was how sensitive the firing behavior is to small changes in parameters such as the threshold or input current. A small increase in current could suddenly cause the neuron to begin spiking, which helped me better understand how neurons act as decision-making units that respond only when enough input has accumulated.